CMSC132: Inheritance

Chapter 2

By Naren Ayinala

May 2026

Contents

1 Inheritance: Reusing and Specializing Classes **2**

1.1 extends, super, and this 2

2 Overloading and Overriding **2**

2.1 Overloading: Compile-Time Selection 2

2.2 Overriding: Runtime Selection 3

2.3 final Methods, Variables, and Classes 3

3 Access Modifiers and Encapsulation **3**

4 Object, References, and Polymorphism **4**

4.1 Early vs Late Binding 4

4.2 Object Equality and the equals Contract 4

4.3 Casting and instanceof 4

5 Abstract Methods and Abstract Classes **5**

6 Composition and Multiple Inheritance **5**

6.1 Hierarchy and "Faking Multiple Inheritance 5

6.2 Practice: Predict the Output 6

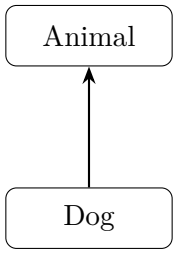
1 Inheritance: Reusing and Specializing Classes

Inheritance describes an *is-a* relationship: a `Dog` is an `Animal`. The child (subclass) inherits accessible members from the parent (superclass), can add behavior, and can override inherited instance methods. It does not inherit constructors. Prefer inheritance for true substitutability rather than merely to reuse a few lines of code.

1.1 extends, super, and this

```
class Animal {
    protected final String name;
    Animal(String name) { this.name = name; }
    void speak() { System.out.println("sound"); }
}
class Dog extends Animal {
    Dog(String name) { super(name); }
    @Override void speak() {
        super.speak(); // invoke superclass implementation
        System.out.println(name + " barks");
    }
}
new Dog("Pip").speak(); // sound, then Pip barks
```

`super(...)` invokes a superclass constructor and must be the first constructor statement if written explicitly. If omitted, Java attempts `super()` automatically. `super.method()` invokes the parent's method; `this.method()` invokes a method on the current object, subject to dynamic dispatch.



2 Overloading and Overriding

These sound similar but behave very differently. **Overloading** means the same method name with different parameter lists; the compiler chooses a signature based on declared types. **Overriding** means a subclass supplies a new implementation of an inherited instance method with a compatible signature; the runtime chooses the implementation based on the actual object.

2.1 Overloading: Compile-Time Selection

```
class Printer {
    void print(int x) { System.out.println("int " + x); }
    void print(String x) { System.out.println("text " + x); }
}
Printer p = new Printer();
p.print(3); // int 3
p.print("hi"); // text hi
```

Changing only a return type does not create a valid overload. Widening, boxing, and varargs can affect overload resolution; therefore avoid ambiguous overload sets.

2.2 Overriding: Runtime Selection

```
class Parent {
    void greet() { System.out.println("parent"); }
}
class Child extends Parent {
    @Override void greet() { System.out.println("child"); }
}
Parent x = new Child();
x.greet(); // child, because the actual object is a Child
```

A subclass cannot reduce an overridden method's visibility. Static methods are hidden rather than overridden, and private methods are not inherited for overriding. Use `@Override` to let the compiler catch mistakes.

2.3 final Methods, Variables, and Classes

A `final` method cannot be overridden; a `final` class cannot be subclassed. A `final` variable can be assigned only once. If the variable holds a reference, the reference cannot be reassigned, but the referenced object may still be mutable.

```
final class ImmutableTag { }
class Base {
    final void identity() { System.out.println("fixed"); }
}
final ArrayList<String> names = new ArrayList<>();
names.add("Ada"); // legal: mutate the object
// names = new ArrayList<>(); // illegal: reassign final variable
```

3 Access Modifiers and Encapsulation

Modifier	Same package	Different package subclass
public	yes	yes
protected	yes	yes, with restrictions
package-private	yes	no
private	same class only	no

Package-private means writing no modifier. A `protected` member is also visible within its own package; outside that package, a subclass's access is restricted to inherited access through the subclass type. Private fields are generally preferred; expose validated operations instead of allowing callers to change state arbitrarily. Prefixes such as `m_` or leading underscores are conventions, not Java requirements.

4 Object, References, and Polymorphism

Every Java class ultimately extends `Object`. Important methods include `toString`, `equals`, and `hashCode`; `getClass` and `wait/notify` also come from `Object`. A superclass variable may hold a subclass instance (**upcasting**). The reference's declared type controls which members are accessible at compile time, while the object's runtime type controls which overridden instance method executes.

4.1 Early vs Late Binding

```
class Animal { void speak() { System.out.println("animal"); } }
class Cat extends Animal {
    @Override void speak() { System.out.println("meow"); }
    void scratch() { System.out.println("scratch"); }
}
Animal a = new Cat();
a.speak(); // meow: dynamic dispatch
// a.scratch(); // compile error: Animal has no scratch()
((Cat) a).scratch(); // valid downcast here
```

The method *signature* is selected using compile-time information (for overloaded methods); the implementation of an overridden instance method is selected dynamically. Fields are not polymorphic: field access depends on the reference's declared type.

4.2 Object Equality and the equals Contract

```
import java.util.Objects;
class Person {
    private final String id;
    Person(String id) { this.id = id; }
    @Override public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Person)) return false;
        Person other = (Person) obj;
        return Objects.equals(id, other.id);
    }
    @Override public int hashCode() { return Objects.hash(id); }
}
```

`==` compares references for objects, while `equals` should implement logical equality. If two objects are equal, their hash codes must match. Equality should be reflexive, symmetric, transitive, consistent, and return false for null. This matters in `HashSet` and `HashMap`.

4.3 Casting and instanceof

Upcasting is safe: every `Cat` is an `Animal`. Downcasting may throw `ClassCastException`; use `instanceof` before casting. Casting does not transform the underlying object into a different class.

```
Animal a = new Cat();
if (a instanceof Cat) {
    Cat c = (Cat) a;
    c.scratch();
}
// Cat impossible = (Cat) new Animal(); // runtime failure
```

5 Abstract Methods and Abstract Classes

An abstract class cannot be instantiated directly. It may contain concrete methods, constructors, instance state, and abstract methods without bodies. A concrete subclass must implement inherited abstract methods, unless it too is abstract.

```
abstract class Shape {
    abstract double area();
    void describe() { System.out.println("area = " + area()); }
}
class Circle extends Shape {
    private final double radius;
    Circle(double r) { radius = r; }
    @Override double area() { return Math.PI * radius * radius; }
}
Shape s = new Circle(2);
s.describe(); // approximately 12.566
```

6 Composition and Multiple Inheritance

Composition is a *has-a* relationship: a `Car` has an `Engine`. It often gives more flexibility than inheritance because a component can be replaced without changing the class hierarchy. Java does not allow a class to extend two classes (avoids conflicting inherited implementations), but it can implement multiple interfaces.

```
interface Flyable { void fly(); }
interface Swimmable { void swim(); }
class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("flying"); }
    public void swim() { System.out.println("swimming"); }
}
class Engine { void start() { System.out.println("started"); } }
class Car {
    private final Engine engine = new Engine(); // has-a
    void drive() { engine.start(); }
}
```

6.1 Hierarchy and "Faking Multiple Inheritance

An inheritance hierarchy has a single class superclass at each step, but a class can implement many interfaces. A practical alternative to multiple implementation inheritance is **composition plus delegation**: store collaborator objects and forward work to them. If two interfaces provide the same default method, the implementing class must resolve the conflict explicitly.

```
interface A { default void show() { System.out.println("A"); } }
interface B { default void show() { System.out.println("B"); } }
class Both implements A, B {
    @Override public void show() { A.super.show(); }
}
```

6.2 Practice: Predict the Output

Given `Parent p = new Child()`, calling an overridden instance method uses the child's implementation, while calling an overloaded method is resolved using declared argument types. A `final` method cannot be overridden. An abstract class may have a constructor. Ask yourself: *is-a* or *has-a*? That usually tells you whether inheritance or composition is appropriate.